SGBD · ANUL II · 2026–2027

Tutoriat 1

Bun venit! Recapitulare SQL înainte de PL/SQL

Ciocirlan Cezar · tutore

```
SELECT * FROM tutoriat;
```

DESPRE TUTORIAT

Ce facem aici

Explicăm pe înțeles

Teoria din curs, fără jargon, cu exemple pe schema HR.

Scriem cod împreună

Live în SQL Developer: pornim de la zero și construim pas cu pas.

Exersăm pentru examen

Exerciții tip examen la fiecare întâlnire și două simulări la final.

Nu există întrebări proaste. Întrerupeți-mă oricând ceva nu e clar.

PLANUL SEMESTRULUI

Drumul până la examen: 10 întâlniri

01
Kickoff + SQL
azi: setup și recapitulare

02
Bazele PL/SQL
blocuri, variabile, IF, bucle

03
Colecții
RECORD, tablouri, vectori

04
Cursoare
parcurgem rezultate linie cu linie

05
Subprograme
proceduri și funcții

06
Excepții
ce faci când ceva merge prost

07
Triggeri
cod care pornește singur

08
Pachete
totul împachetat la un loc

09
Simulare 1
examen de probă, cu corectare

10
Simulare 2
ultima repetiție

EVALUARE

Examenul: ce știm și ce mai aflăm

Ce știm

- Examen: 8 p, inclusiv 1 p din oficiu.
- Activitate: minimum 1 p la laborator și 1 p la curs, care nu se recuperează.
- Fără activitate, nota maximă e 8.
- Bonusurile de la curs și laborator se adaugă la examen.

De aflat de la titular

Ce	Răspuns
Format	[hârtie / calculator]
Durată	[__ minute]
Materiale permise	[__]
Tipuri de subiecte	[__]

DE CE PL/SQL?

SQL spune ce vrei.
PL/SQL spune și cum.

PL/SQL = Procedural Language extension to SQL. Peste SQL adaugă variabile, condiții, bucle, proceduri și triggeri.

VOCABULAR

Trei termeni, pe scurt

SGBD

Programul care stochează și gestionează datele.

ex: Oracle, PostgreSQL, MySQL

SQL

Limbajul cu care ceri sau modifici date. Spui ce vrei, nu cum.

ex: SELECT, INSERT, UPDATE

PL/SQL

Extensia procedurală Oracle: SQL plus variabile, condiții, bucle.

ex: DECLARE … BEGIN … END;

Blocul tău PL/SQL
IF, LOOP, variabile + SQL

→

Motorul PL/SQL
execută partea procedurală

→

Motorul SQL (serverul)
execută SELECT, UPDATE…

MEDIUL DE LUCRU · ORACLE 19C

Instanță, bază de date, containere

Instanța · în memorie

SGA, PARTAJATĂ DE TOȚI

buffer cache: blocuri de date
shared pool: cod compilat, dicționar
redo log buffer: modificările recente

PROCESE DE FUNDAL

`DBWn · LGWR · CKPT · SMON · PMON`



Baza de date · pe disc

fișiere de date: tabelele, indecșii
redo log: jurnalul modificărilor
control file: harta bazei
fișierul de parametri

CDB$ROOT

containerul rădăcină: dicționarul comun

PDB, ex. ORCLPDB

schema HR: aici lucrați voi

PDB$SEED

șablon pentru PDB-uri noi

PREGĂTIRE

Setup în 5 pași

1. Instalează SQL Developer (arhiva e pe Teams).
2. Creează conexiunea cu datele primite la laborator.
3. Deschide hr_create.sql și rulează-l cu F5.
4. Verifică: în employees trebuie să fie 107 linii.
5. Pornește afișarea: SET SERVEROUTPUT ON.

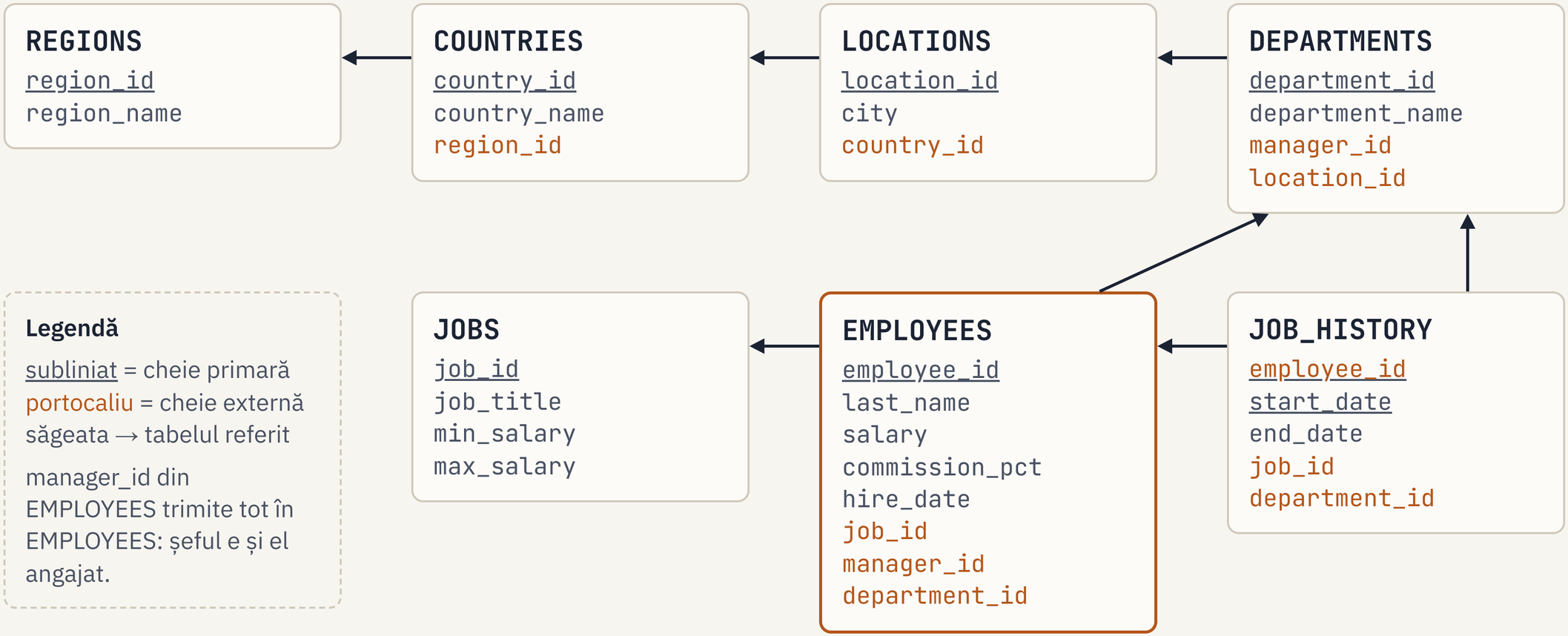
```
-- verificare după hr_create.sql
SELECT COUNT(*) FROM employees;  -- 107
-- copia de lucru, cu sufixul tău
CREATE TABLE emp_pnu AS
  SELECT * FROM employees;

SET SERVEROUTPUT ON
```

F5 rulează tot scriptul. **Ctrl+Enter** rulează doar instrucțiunea pe care stă cursorul.

SCHEMA HR

Harta pe care lucrăm tot semestrul



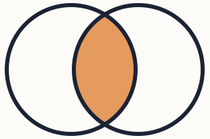
Legendă

subliniat = cheie primară
portocaliu = cheie externă
săgeata → tabelul referit

manager_id din EMPLOYEES trimite tot în EMPLOYEES: șeful e și el angajat.

Tipuri de JOIN: pe cine păstrăm?

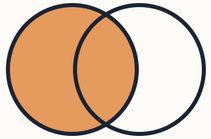
A = employees (107 angajați) · B = departments (27 de departamente)



INNER JOIN

doar perechile care se potrivesc

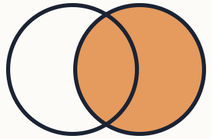
106 linii: K. Grant dispare, n-are departament



LEFT JOIN

tot din A, plus potrivirile din B

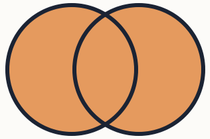
107 linii: K. Grant apare cu departament NULL



RIGHT JOIN

tot din B, plus potrivirile din A

122 linii: apar și cele 16 departamente goale



FULL JOIN

tot din A și tot din B

123 linii: și K. Grant, și departamentele goale

A LEFT JOIN B dă același rezultat ca B RIGHT JOIN A. Contează doar ordinea tabelelor.

RECAPITULARE SQL · CAPCANĂ

LEFT JOIN + WHERE = INNER JOIN deghizat

⚠ Greșit

```
SELECT e.last_name, d.department_name
FROM employees e
LEFT JOIN departments d
  ON e.department_id = d.department_id
WHERE d.location_id = 1700;
-- K. Grant dispare: location_id e NULL
```

✓ Corect

```
SELECT e.last_name, d.department_name
FROM employees e
LEFT JOIN departments d
  ON e.department_id = d.department_id
 AND d.location_id = 1700;
-- toți cei 107 angajați rămân
```

⚠ O condiție în WHERE pe tabelul din dreapta aruncă liniile cu NULL, deci LEFT JOIN-ul devine INNER JOIN. Pune condiția în ON.

Un tabel, două roluri

```sql
SELECT e.last_name AS angajat,
       m.last_name AS sef
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.employee_id;
```

angajat	sef
Kochhar	King
De Haan	King
Fay	Hartstein
King	NULL

1. Același tabel, cu două aliasuri: **e** e angajatul, **m** e șeful.
2. Legătura: manager_id-ul angajatului e employee_id-ul șefului.
3. LEFT JOIN, ca să apară și King, care nu are șef.

> Imaginează-ți două copii ale tabelului puse una lângă alta.

Subcereri corelate

Subcererea folosește o coloană din cererea de afară, deci se recalculează pentru fiecare linie.

```sql
-- angajații care câștigă peste media departamentului lor
SELECT last_name, salary, department_id
FROM employees e
WHERE salary > (SELECT AVG(salary)
                FROM employees
                WHERE department_id = e.department_id);
```

1 · Ia un angajat

Hartstein, departamentul 20, salariu 13000

2 · Rulează subcererea

media departamentului 20 = 9500

3 · Compară

13000 > 9500: Hartstein rămâne. Fay (6000) nu.

RECAPITULARE SQL · INTEROGĂRI

Ordinea de execuție a unui SELECT

Oracle nu citește cererea de sus în jos: o evaluează în alt ordin decât îl scriem.

```
SELECT d.department_name, AVG(e.salary) AS medie      5
FROM employees e                                      1
JOIN departments d USING (department_id)              1
WHERE e.job_id <> 'AD_PRES'                           2
GROUP BY d.department_name                            3
HAVING AVG(e.salary) > 8000                           4
ORDER BY medie DESC;                                  6
```

Consecință: aliasul **medie** se creează la pasul 5, deci nu poate fi folosit în WHERE sau HAVING, dar poate fi folosit în ORDER BY (pasul 6).

1. **FROM + JOIN** — alege și îmbină tabelele
2. **WHERE** — filtrează liniile
3. **GROUP BY** — formează grupurile
4. **HAVING** — filtrează grupurile
5. **SELECT** — calculează coloanele
6. **ORDER BY** — sortează rezultatul

WHERE vs HAVING: înainte și după grupare

FROM tabelele → **WHERE** filtrează linii → **GROUP BY** grupează → **HAVING** filtrează grupuri → **SELECT** coloanele → **ORDER BY** sortează

```
SELECT department_id, AVG(salary) AS medie
FROM employees
WHERE job_id <> 'AD_PRES'    -- linii
GROUP BY department_id
HAVING AVG(salary) > 8000;   -- grupuri
```

⚠ AVG, SUM, COUNT nu au voie în WHERE: acolo încă nu există grupuri.

💡 HAVING fără GROUP BY? Merge: tot tabelul devine un singur grup (Q9).

Primii 5 cei mai bine plătiți

Greșit

```
SELECT last_name, salary
FROM employees
WHERE ROWNUM <= 5
ORDER BY salary DESC;
-- ia 5 linii oarecare,
-- abia apoi le sortează
```

Corect: subcerere + ROWNUM

```
SELECT * FROM (
  SELECT last_name, salary
  FROM employees
  ORDER BY salary DESC
)
WHERE ROWNUM <= 5;
```

Corect, din Oracle 12c

```
SELECT last_name, salary
FROM employees
ORDER BY salary DESC
FETCH FIRST 5 ROWS ONLY;
```

Salarii egale la limită? FETCH FIRST 5 ROWS **WITH TIES** îi păstrează și pe cei la egalitate cu al 5-lea.

Clasamente, fără să pierzi liniile

```
SELECT last_name, salary,
  ROW_NUMBER() OVER (ORDER BY salary DESC) rn,
  RANK()       OVER (ORDER BY salary DESC) rk,
  DENSE_RANK() OVER (ORDER BY salary DESC) drk
FROM employees
WHERE department_id = 60;
```

Nume	Salariu	rn	rk	drk
Hunold	9000	1	1	1
Ernst	6000	2	2	2
Austin	4800	3	3	3
Pataballa	4800	4	3	3
Lorentz	4200	5	**5**	**4**

Cu PARTITION BY department_id, clasamentul o ia de la capăt în fiecare departament. O funcție analitică nu poate sta în WHERE: filtrezi rezultatul dintr-o subcerere.

Arborele șefilor cu CONNECT BY

```
SELECT LEVEL,
       LPAD(' ', 2*(LEVEL-1)) || last_name AS ierarhie
FROM employees
START WITH manager_id IS NULL
CONNECT BY PRIOR employee_id = manager_id;
```

START WITH: de unde pornim. Aici, de la King, care nu are șef.

CONNECT BY PRIOR employee_id = manager_id: cobori de la un șef la subordonații lui.

LEVEL: nivelul în arbore (1 = rădăcina).

```
1  King
2    Kochhar
3      Greenberg
2    De Haan
3      Hunold
```

Cererile cu „pe toate”

Angajații care au lucrat pe toate proiectele cu buget 10000.

1 · Numeri și compari

```sql
SELECT employee_id
FROM works_on
WHERE project_id IN (SELECT project_id
  FROM projects WHERE budget = 10000)
GROUP BY employee_id
HAVING COUNT(DISTINCT project_id) =
  (SELECT COUNT(*) FROM projects
   WHERE budget = 10000);
```

2 · Dublă negație

```sql
SELECT DISTINCT a.employee_id
FROM works_on a
WHERE NOT EXISTS (
  SELECT 1 FROM projects p
  WHERE p.budget = 10000 AND NOT EXISTS (
    SELECT 1 FROM works_on b
    WHERE b.project_id = p.project_id
    AND b.employee_id = a.employee_id));
```



Varianta 2 se citește: „nu există proiect cu buget 10000 pe care angajatul să nu fi lucrat”.

QUIZ · 1

Care este cheia primară?

cod	nume	info
11	Emil	NULL
22	Dana	DA
13	NULL	NU
11	Marius	NULL
44	Emil	DA

Alegeți coloana (sau coloanele) care poate fi cheie primară și justificați.

Cheie primară = valori **unice** și **niciodată NULL**.

QUIZ · 2

Ce returnează cererea?

```
SELECT department_id, hire_date, SUM(salary)
FROM employees
GROUP BY department_id, hire_date;
```

A

Suma salariilor pentru fiecare departament

B

Suma salariilor pentru fiecare pereche (departament, dată de angajare)

C

Eroare: o dată calendaristică nu poate sta în GROUP BY

EXERCIȚII

De lucrat acum și acasă

Nr.	Nivel	Cerință
1	*	Pentru fiecare oraș: țara în care se află și numărul de angajați.
4	*	Departamentele fără angajați (două soluții: cu join și fără).
6	*	Angajații care au ocupat de mai multe ori aceeași funcție.
2	**	Numele, salariul, jobul, orașul și țara subordonaților direcți ai lui King.
15	**	Angajații care au unul dintre cele mai mari 10 salarii.
16	***	Pentru fiecare angajat: câți oameni conduce, ierarhic, nu doar direct.

Toate sunt din Questions.pdf și rulează pe hr_create.sql. Soluțiile le primiți după întâlnire.

DATA VIITOARE

Bazele PL/SQL

Blocuri, variabile, IF și bucle. Primul nostru cod care „gândește”.

Întrebări?

PÂNĂ ATUNCI

- ✓ SQL Developer instalat și conectat
- ✓ hr_create.sql rulat: 107 angajați
- ✓ Exercițiile 2, 15 și 16 din Questions
- ☆ Bonus: un bloc care afișează „Hello World” (Curs 2)